Object-Aware Latent Adversarial Attack on YOLOv8

Mathematical formulation of the method

Mohamed Aziz Brahmi
PFE – Adversarial robustness of object detectors

May 5, 2026

Abstract

This document gives the full mathematical formulation of the attack implemented in `src/attack.py`. The README contains a one-paragraph summary; this is the formal version used in the report.

Note pour l'encadrant

Ce document est la version detaillee de la methode. Il est organise en 14 sections courtes : la section 1 fixe les notations, les sections 3–8 construisent les briques (masques, forward, fonction objectif, contraintes), la section 9 donne l'algorithme d'optimisation complet, et les sections 10–14 discutent les choix de modelisation et les limites. Les encadres bleus "Note pour l'encadrant" sont des explications ajoutees pour ce document uniquement et n'apparaitront pas tels quels dans le rapport final.

1 Notation

Symbol	Space	Meaning
x	$\mathbb{R}^{3\times H\times W}$, values in $[0,1]$	clean RGB image
f	$f:\mathbb{R}^{3\times H\times W}\to\mathbb{R}^{N\times(4+C)}$	frozen YOLOv8 detector (pre-NMS head)
$E,\ D$	SD-VAE encoder / decoder	frozen Stable-Diffusion VAE
$z=E(x)$	$\mathbb{R}^{4\times H/8\times W/8}$	latent encoding of x
δ	$\mathbb{R}^{4\times H/8\times W/8}$	learnable latent perturbation
M	$\{0,1\}^{H\times W}$	pixel mask = union of detected boxes
M_z	$\{0,1\}^{H/8\times W/8}$	latent mask (pooled M)
$\mathcal{D}_{\text{clean}}$	set of detections	YOLOv8 outputs on x after NMS, conf $>\tau$
$\mathcal{C}_{\text{clean}}$	subset of $\{0,\dots,C-1\}$	classes that appear in $\mathcal{D}_{\text{clean}}$
ε_z	scalar	L_∞ budget on δ
γ	scalar in $(0,1)$	confidence floor in the vanishing loss
τ	scalar	NMS confidence threshold (default 0.25)

Image tensors are in $[0,1]$; the VAE expects inputs in $[-1,1]$, so E and D internally apply the affine $2x-1$ and $(y+1)/2$.

2 Threat model

- **White-box, digital, untargeted (per class).** The attacker has full access to the frozen detector f and the frozen VAE (E,D). No weights are updated. The attacker observes $\mathcal{D}_{\text{clean}}$ once and tries to drive every class in $\mathcal{C}_{\text{clean}}$ below the detection threshold.

- **Image-conditional, instance-agnostic.** The perturbation δ is optimized per image. We do not require persistent identities across frames; success is measured frame-by-frame.
- **Locality.** The pixel-space change is restricted to M (the union of clean boxes) by a paste-back operator. The latent change is restricted to M_z by elementwise masking of δ.
- **No physical realizability.** This is a digital attack; we make no claim about printability, illumination invariance, or perspective robustness.

Note pour l'encadrant

On choisit un cadre *white-box* car il fournit la borne superieure de ce qu'un attaquant peut realiser : si la defense resiste ici, elle resiste *a fortiori* en black-box. Le cadre *digital* est impose par l'objectif du PFE (etudier la robustesse algorithmique du detecteur, pas la faisabilite physique d'une attaque routiere). *Untargeted per class* signifie qu'on cherche a faire disparaitre les detections, pas a les re-etiqueter – c'est l'attaque qui maximise les faux negatifs.

3 Bounding-box masks

Let $\mathcal{D}_{\text{clean}} = \{(b_i, c_i, p_i)\}_{i=1}^{K}$ with $b_i = (x_1, y_1, x_2, y_2)$ in pixel coordinates. The pixel mask is the union of clean boxes:

$$M[h, w] = \begin{cases} 1 & \text{if } \exists\, i \text{ s.t. } (h, w) \in b_i, \\ 0 & \text{otherwise.} \end{cases} \tag{1}$$

The latent mask is obtained by an 8×8 max-pool with stride 8 – i.e. a latent cell is "inside" the perturbable region iff *any* of its 8×8 pixel cells lies in M:

$$M_z = \text{MaxPool}_{\text{kernel}=8,\ \text{stride}=8}(M). \tag{2}$$

This guarantees that every pixel inside a clean box is covered by at least one perturbable latent cell, while keeping the perturbable region as tight as possible.
M and M_z are computed once per image and held constant for the rest of the optimization.

Note pour l'encadrant

Le choix de *max-pool* (et non average-pool) est important : on veut une *couverture conservative*, c'est-a-dire ne pas exclure du masque latent une cellule qui contient ne serait-ce qu'un pixel d'objet. Si on utilisait average-pool avec un seuil, on risquerait des trous au bord des boites. Le facteur 8 vient de la profondeur du VAE de Stable Diffusion (downsampling $\times 8$) et coincide avec le *stride* du premier niveau de detection de YOLOv8 – c'est cette coincidence qui rend la perturbation latente naturellement aligne avec la grille de decision du detecteur.

4 Forward pass (clean and adversarial)

Encoding is done once, with no gradient:

$$z = E(x) \quad \text{(detached)}. \tag{3}$$

Given a perturbation δ, the adversarial latent and decoded image are

$$z_{\text{adv}} = z + (M_z \odot \delta), \tag{4}$$
$$x_{\text{dec}} = D(z_{\text{adv}}), \quad \text{(autograd-friendly)} \tag{5}$$
$$x_{\text{adv}} = M \odot x_{\text{dec}} + (1 - M) \odot x. \tag{6}$$

The paste-back operator on the last line (6) is the key locality device: pixels outside the union of clean boxes are byte-identical to the input. Only pixels inside M can change, regardless of how D reconstructs the rest of the image.
The adversarial detector output (pre-NMS) is

$$y = f(x_{\text{adv}}) \in \mathbb{R}^{N\times(4+C)}, \tag{7}$$

where the last C channels of each anchor are class confidences in $[0, 1]$ (post-sigmoid in YOLOv8's head).

Note pour l'encadrant

Le *paste-back* est ce qui distingue notre methode d'une attaque latente naive. Un VAE ne reconstruit jamais exactement l'image d'entree (*reconstruction loss* non nulle) ; sans paste-back, le simple fait d'encoder puis decoder l'image clean introduit deja des artefacts hors des boites, ce qui : (i) contamine la metrique PSNR globale, (ii) rend impossible de claimer que la perturbation est localisee aux objets. Avec paste-back, on garantit *par construction* que seuls les pixels couverts par M peuvent differer de x.

5 Vanishing loss (class-level max-confidence)

For each class c originally present in $\mathcal{C}_{\text{clean}}$, define its image-level max confidence

$$p_c(x_{\text{adv}}) = \max_{n\in\{1,\dots,N\}} y[n,\ 4+c]. \tag{8}$$

The vanishing loss penalizes any class whose max confidence is still above the floor γ:

$$\mathcal{L}_{\text{vanish}}(x_{\text{adv}}) = \sum_{c\in\mathcal{C}_{\text{clean}}} \max\big(p_c(x_{\text{adv}}) - \gamma,\ 0\big). \tag{9}$$

This formulation is intentionally **class-level** rather than **instance-level**: we do not match adversarial anchors to clean boxes via IoU, we simply require that no anchor anywhere in the image predicts class c with confidence above γ. This sidesteps the non-differentiability of NMS and the brittleness of IoU matching, and empirically suffices to remove every detection of every targeted class under standard YOLOv8 NMS at threshold $\tau \geq \gamma$.

The hinge with floor γ (rather than a plain p_c term) prevents the optimizer from wasting budget pushing already-suppressed classes further down.

Note pour l'encadrant

Equation (9) est le coeur de la methode. Trois remarques :

1. Le max sur les anchors est sous-differentiable mais pas un *argmax* discret : PyTorch propage le gradient sur l'anchor maximal, ce qui suffit en pratique (technique standard en attaque YOLO).
2. Le *hinge* ($\max(\cdot - \gamma, 0)$) joue le role d'un seuil de « satisfait » : une fois $p_c < \gamma$, la classe c ne contribue plus au gradient, et le budget se reporte sur les classes encore visibles.
3. La sommation sur $\mathcal{C}_{\text{clean}}$ (pas sur toutes les classes) evite de creer accidentellement de nouvelles detections de classes absentes a l'origine.

6 Perceptual and regularization terms

To keep the perturbation visually unobtrusive we add a masked L_2 term in pixel space, normalized by the masked area:

$$\mathcal{L}_{\text{perc}}(x_{\text{adv}}, x) = \frac{\|M \odot (x_{\text{adv}} - x)\|_2^2}{3 \cdot \|M\|_1}. \tag{10}$$

$\|M\|_1$ is the number of foreground pixels; the factor 3 accounts for RGB channels. Normalizing makes λ_p interpretable independently of how much of the frame is covered by boxes.
We also penalize the magnitude of δ directly – a soft prior that keeps latents near the clean encoding even when the budget is loose:

$$\mathcal{L}_{\text{reg}}(\delta) = \frac{\|M_z \odot \delta\|_2^2}{\|M_z\|_1}. \tag{11}$$

7 Objective

$$\boxed{\mathcal{L}(\delta) = \mathcal{L}_{\text{vanish}}(x_{\text{adv}}) + \lambda_p \cdot \mathcal{L}_{\text{perc}}(x_{\text{adv}}, x) + \lambda_r \cdot \mathcal{L}_{\text{reg}}(\delta)} \tag{12}$$

with x_{adv} defined in section 4 as a function of δ.
Default weights (see `configs/default.yaml`): $\lambda_p = 0.05$, $\lambda_r = 10^{-3}$. The vanishing term is the only term that drives the attack; the other two are taste-makers.

8 Constraint and projection

The perturbation lives in an L_∞ ball of radius ε_z in latent space:

$$\delta \in B_\infty(\varepsilon_z) = \left\{ d \ : \ \|d\|_\infty \le \varepsilon_z \right\}. \tag{13}$$

We additionally restrict δ to the latent mask M_z (zero outside the box footprint, all the time).

After every Adam step we project:

$$\delta \ \leftarrow \ M_z \odot \text{clip}(\delta, \ -\varepsilon_z, \ +\varepsilon_z). \tag{14}$$

We do **not** clip x_{adv} to $[0, 1]$ between steps – the loss already penalizes deviation from x and the paste-back operator only affects pixels inside M. We do clip x_{adv} to $[0, 1]$ once at the end before saving.

9 Optimization

Algorithm 1 – Object-aware latent adversarial attack

Input: clean image x, detector f, VAE (E, D), hyperparameters $\varepsilon_z, \gamma, \lambda_p, \lambda_r$, lr, T.
Output: adversarial image x_{adv}, perturbation δ.

---

1. Run NMS on $f(x)$ to obtain $\mathcal{D}_{\text{clean}}$ and $\mathcal{C}_{\text{clean}}$.
2. Compute pixel mask M and latent mask M_z (§3).
3. $z \leftarrow E(x)$ (detach gradient).
4. Initialize $\delta \leftarrow \mathbf{0}$.
5. Initialize Adam optimizer with parameter δ and learning rate lr.
6. **For** $t = 1, \ldots, T$ **do**
   (a) $z_{\text{adv}} \leftarrow z + M_z \odot \delta$.

(b) $x_{\text{dec}} \leftarrow D(z_{\text{adv}})$.
(c) $x_{\text{adv}} \leftarrow M \odot x_{\text{dec}} + (1 - M) \odot x$.
(d) $y \leftarrow f(x_{\text{adv}})$.
(e) $\mathcal{L} \leftarrow \mathcal{L}_{\text{vanish}}(y) + \lambda_p \mathcal{L}_{\text{perc}}(x_{\text{adv}}, x) + \lambda_r \mathcal{L}_{\text{reg}}(\delta)$.
(f) Compute $\nabla_\delta \mathcal{L}$ via back-propagation.
(g) Update δ with one Adam step.
(h) Project: $\delta \leftarrow M_z \odot \text{clip}(\delta, -\varepsilon_z, +\varepsilon_z)$ (eq. (14)).
(i) **If** $\forall c \in \mathcal{C}_{\text{clean}},\ p_c(x_{\text{adv}}) < \gamma$ **then break** (early stop on full vanishing).

7. **Return** x_{adv}, δ.

Defaults: $T = 80$ steps, lr $= 0.01$, $\varepsilon_z = 0.10$, $\gamma = 0.05$. Adam is preferred over SGD because the gradient magnitude through $f \circ D$ varies sharply across latent channels.

Note pour l'encadrant

La sortie anticipee a la ligne 17 (*early stop on full vanishing*) est une optimisation : des que toutes les classes originellement detectees sont passees sous γ, l'attaque a reussi et continuer ne ferait qu'augmenter la distorsion. Cela explique pourquoi sur certaines images faciles le compteur d'iterations s'arrete bien avant $T = 80$.

10 Why latent-space perturbation?

Three reasons, each motivated by a known weakness of pixel-space attacks (FGSM, PGD):

1. **Built-in image prior.** The decoder D is a learned manifold of natural-looking images. Updates that move z in a generic direction tend to decode to plausible textures, not high-frequency salt-and-pepper noise. This is what gives latent attacks their characteristic smoother appearance at comparable detection-failure rates.
2. **Object-locality at the right granularity.** YOLOv8's stride at the first detection scale is 8, exactly matching the SD-VAE downscaling factor. A single latent cell corresponds to a single 8×8 image patch – the scale at which YOLO actually makes its decisions. Restricting δ to M_z therefore restricts the attack to the exact receptive-field cells responsible for the boxes we want to remove.
3. **Decoupling the budget from pixel intensities.** Pixel-space ε is a blunt instrument: at $\varepsilon = 8/255$ even a "small" perturbation over a large box is visually obvious as noise. A latent budget with $\varepsilon_z = 0.10$ produces pixel-space changes that the decoder spreads over a structured texture, and the masked L_2 term keeps the visible part bounded explicitly.

Note pour l'encadrant

Le point 2 est l'argument theorique le plus solide pour justifier le choix du *latent-space* : ce n'est pas une coincidence numerique, c'est exactement la granularite a laquelle YOLOv8 prend ses decisions au premier niveau de detection. Cela permet de presenter la methode comme « perturbation a l'echelle decisionnelle du detecteur », plutot que comme « bruit dans un espace abstrait ».

11 Differences from the IoU-matched formulation

An earlier draft of this method matched each adversarial anchor to the nearest clean box by IoU and minimized that anchor's confidence. We replaced this by the class-level max for three

reasons:

- **Differentiability.** arg max over anchors is sub-differentiable; taking the max over the per-class confidence channel is the standard trick used in YOLO adversarial work and gives clean gradients.
- **NMS independence.** The class-level loss does not depend on which anchors survive NMS, so the attack target does not drift between optimization steps as different anchors light up.
- **Empirical equivalence.** On DETRAC, both formulations converged to the same DFR (1.0) and similar PSNR-mask, but the class-level version converges in fewer steps and is materially simpler.

12 Evaluation metrics

(Defined here, computed in `scripts/evaluate.py`.)

Metric	Definition
DFR (detection-failure rate)	fraction of frames with $\mathcal{D}_{\text{adv}} = \emptyset$, where $\mathcal{D}_{\text{adv}} = \text{NMS}(f(x_{\text{adv}}), \tau, \text{iou\_thr})$
ASR (attack success rate)	fraction of frames in which every class originally present is fully removed (i.e. $\mathcal{C}_{\text{clean}} \cap \text{classes}(\mathcal{D}_{\text{adv}}) = \emptyset$)
mean conf drop	average over $c \in \mathcal{C}_{\text{clean}}$ of $p_c(x) - p_c(x_{\text{adv}})$
$\textbf{PSNR}_{\text{mask}}$	PSNR computed only over pixels inside M, in dB; higher = stealthier
masked L_2	$\|M \odot (x_{\text{adv}} - x)\|_2 / \sqrt{3 \cdot \|M\|_1}$ – average per-pixel L_2 inside the mask

For aggregate reporting on a set of frames we report the mean of each metric and the standard error, plus a pixel-budget-matched comparison against FGSM and PGD baselines (`baselines/fgsm.py`, `baselines/pgd_pixel.py`).

Note pour l'encadrant

Les deux metriques de succes (DFR et ASR) capturent deux severites differentes : DFR exige que le detecteur ne renvoie *rien*, ASR exige seulement que les classes *originellement presentes* disparaissent. ASR $\geq$ DFR par construction. La distinction PSNR *globale* vs $\text{PSNR}_{\text{mask}}$ est cruciale : l'image entiere est en grande partie inchangee (paste-back), donc la PSNR globale est artificiellement elevee. Seule la PSNR sur le masque mesure honnetement la perceptibilite de la perturbation.

13 Reproducibility

The full pipeline is deterministic up to CUDA non-determinism in the detector backbone:

- Seeds are set in `src/utils.set_seed` for `torch`, `numpy`, and Python's `random`.
- VAE weights: `stabilityai/sd-vae-ft-mse` (frozen).
- Detector weights: `runs/yolov8n_detrac/best.pt` (trained on DETRAC, see `notebooks/train_yolov8_detr`
- All hyperparameters live in `configs/default.yaml`.
- Per-image attack metadata (initial detections, final detections, number of steps, final loss components) is written to `results/<run>/metadata/<image>.json` by `scripts/run_attack.py`.

14 Limitations

- **No physical-world claims.** The attack is digital; printing $x_{\text{adv}} - x$ and pasting it onto a real vehicle is out of scope.
- **No transfer claims.** We attack the specific YOLOv8n detector trained on DETRAC. Transfer to other detectors (e.g. YOLOv5, RT-DETR) or other domains is not evaluated.
- **NMS dependence.** "Detection failure" is defined relative to the configured $(\tau, \text{iou\_thr})$. A defender who lowers τ may recover some objects.
- **Per-image cost.** Optimization is ~ 80 backward passes through $f \circ D$, which is two orders of magnitude more expensive than FGSM. Real-time use is not the goal.

Note pour l'encadrant

Cette section des limites est intentionnellement honnete et explicite : elle anticipe les questions d'un jury / lecteur critique. Le rapport final reprendra ces points dans une section *Discussion* pour montrer que le scope de l'etude est maitrise.